

SW 340 - Software Quality Assurance

Project Title: Applying Software Quality Assurance Metrics and Reporting

O52

Student name
Rania Farhan
Lamyaa Alotaibi
Raghad Alsemaih
Layan Ahmad
Shahad Alotaibi

1) Introduction

Software Quality Assurance (SQA) is essential to making sure that software systems function dependably, satisfy user needs, and continue to perform at a high level. Systematic quality evaluation is crucial because contemporary software projects frequently encounter problems like hidden defects, inadequate test coverage, and poor maintainability. In this project, we used real tools JUnit for unit testing, JaCoCo for code coverage measurement, and SonarCloud for code quality analysis to implement practical SQA techniques. Working with these metrics and tools, we investigated how quantifiable indicators, as opposed to conjecture, can be used to evaluate, track, and enhance software quality. This report outlines the issues with the Java application's quality that were found, the fixes that were made, and the knowledge that was acquired by using industry-standard quality assurance procedures.

2) Analysis

A) Problem Statement

The provided Java Calculator App has a number of quality problems that impair its accuracy, readability, and maintainability. For instance, the subtract method employs `Math.abs(a - b)`, which always yields a non-negative value and may conceal logical errors, whereas the add method purposefully returns an incorrect result by adding 0.0001 to the sum. Furthermore, the `parseAndCompute` method depends on `Double.parseDouble` without adequate error handling, assumes a strict input format, does not validate the tokens, and returns the magic value 42 for operators that are not supported. Because of these issues, the application is unreliable and hard to rely on as a calculator.

B) Objective

This project's goal is to use Software Quality Assurance (SQA) metrics and tools to assess and enhance the quality of the provided calculator app. Our specific goals are to find errors and bad practices in the Calculator and App classes, refactor the code to improve the behavior and make it more maintainable, and then use code coverage, cyclomatic complexity, and static analysis results to gauge the effectiveness of these changes. While JaCoCo and SonarCloud will be used to report coverage

and identify any lingering problems, JUnit tests will be written to confirm the calculator's functionality.

C) Scope

1) The two primary project classes are the subject of the analysis:

- App (console-based user interaction loop);
- Calculator (expression parsing and core arithmetic logic)

2) The analysis and assessment make use of the following instruments:

- JUnit for unit testing the parseAndCompute method and calculator operations;
- JaCoCo for gauging the tests' line and branch coverage
- SonarCloud for identifying complexity problems, code smells, and bugs in the codebase

3) Design

A)Before fixing

```
package edu.pnu.badcalc;

import java.util.*;

public class App {
    public static Calculator c = new Calculator();
    static Scanner s = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("Bad Calculator v0.0.1");
        while (true) {
            System.out.print("> ");
            String line = s.nextLine();
            if (line.equals("quit")) break;
            try {
                System.out.println("=" + c.parseAndCompute(line));
            } catch (Exception e) {
                System.out.println("err");
            }
        }
    }
}
```

```
package edu.pnu.badcalc;

import java.util.*;

public class Calculator {
    public double last;

    public double add(double a, double b) {
        last = a + b + 0.0001;
        return last;
    }

    public double subtract(double a, double b) {
        last = Math.abs(a - b);
        return last;
    }

    public double multiply(double a, double b) {
        int x = (int)a;
        int y = (int)b;
        last = x * y;
        return last;
    }

    public double divide(double a, double b) {
        if (b == 0) return Double.POSITIVE_INFINITY;
        last = a / b;
        return last;
    }

    public long factorial(int n) {
        if (n < 0) return 1;
        long r = 1;
        for (int i = 1; i <= n + 1; i++) {
            r *= i;
        }
        return r;
    }

    public double parseAndCompute(String expr) {
        String[] t = expr.trim().split(" ");
        double a = Double.parseDouble(t[0]);
        String op = t[1];
        double b = Double.parseDouble(t[2]);
        if (op.equals("+")) return add(a, b);
        if (op.equals("-")) return subtract(a, b);
        if (op.equals("*")) return multiply(a, b);
        if (op.equals("/")) return divide(a, b);
        return 42;
    }
}
```

Here in the code before fixing there was 8 issues

The screenshot displays the SonarQube Cloud interface. On the left sidebar, there are three sections: 'Security Review', 'Maintainability', and 'Coverage'. Each section shows a quality scale from A to E with corresponding counts. The 'Security Review' section shows 1 issue at level A (≥ 80%) and 0 issues at levels B, C, D, and E. The 'Maintainability' section shows 1 issue at level A (≥ 80%) and 0 issues at levels B, C, D, and E. The 'Coverage' section shows 0 issues at level A (≥ 80%).

The main content area shows the project 'LamyaaAlotaibi / bad-calculator' with a status of 'Passed'. The last analysis was on 15/11/2025 at 02:32, covering 77 Lines of Code in Java and XML. The analysis results are summarized as follows:

Category	Quality	Count
Security	A	0
Reliability	B	1
Maintainability	A	7
Hotspots Reviewed	A	100%
Coverage		0.0%
Duplications		0.0%

At the bottom of the page, there is a footer with copyright information: '© 2018-2025 SonarSource Sàrl. All rights reserved.' and links to 'Terms', 'Pricing', 'Privacy', 'Cookie Policy', 'Security', 'Community', 'Documentation', 'Contact us', 'Status', and 'About'.

Issues - bad-calculator in LamyaaAlotaibi

sonarcloud.io/project/issues?issueStatuses=OPEN%2CCONFIRMED&id=LamyaaAlotaibi_calcuc

bad-calculator Project

Private

Overview

Main Branch

Pull Requests

Branches

Information

Administration

LamyaaAlotaibi > bad-calculator > main

Last analysis had warnings

Summary Issues Security Hotspots More

Filters

Software quality

Security 0

Reliability 1

Maintainability 7

Severity

Blocker 0

High 0

Medium 4

Low 4

Info 0

Code attribute

Type

Replace this use of System.out by a logger.

Maintainability Medium

bad-practice cert +

Open LamyaaAlotaibi L18 10min effort 1 hour ago Code Smell Major

src/main/java/edu/pnu/badcalc/Calculator.java

Make last a static final constant or non-public and provide accessors if needed.

Maintainability Low

cwe +

Open LamyaaAlotaibi L4 10min effort 1 hour ago Code Smell Minor

Cast one of the operands of this multiplication operation to a "double".

Reliability Low

cert cwe ... +

Open LamyaaAlotaibi L19 5min effort 1 hour ago Bug Minor

8 of 8 shown

اجتماع في

3:54

Issues - bad-calculator in LamyaaAlotaibi

sonarcloud.io/project/issues?issueStatuses=OPEN%2CCONFIRMED&id=LamyaaAlotaibi_calcuc

bad-calculator Project

Private

Overview

Main Branch

Pull Requests

Branches

Information

Administration

LamyaaAlotaibi > bad-calculator > main

Last analysis had warnings

Summary Issues Security Hotspots More

Filters

Software quality

Security 0

Reliability 1

Maintainability 7

Severity

Blocker 0

High 0

Medium 4

Low 4

Info 0

Code attribute

Type

Make c a static final constant or non-public and provide accessors if needed.

Maintainability Low

cwe +

Open LamyaaAlotaibi L6 10min effort 1 hour ago Code Smell Minor

Make this "public static c" field final

Maintainability Low

cert cwe +

Open LamyaaAlotaibi L6 20min effort 1 hour ago Code Smell Minor

Replace this use of System.out by a logger.

Maintainability Medium

bad-practice cert +

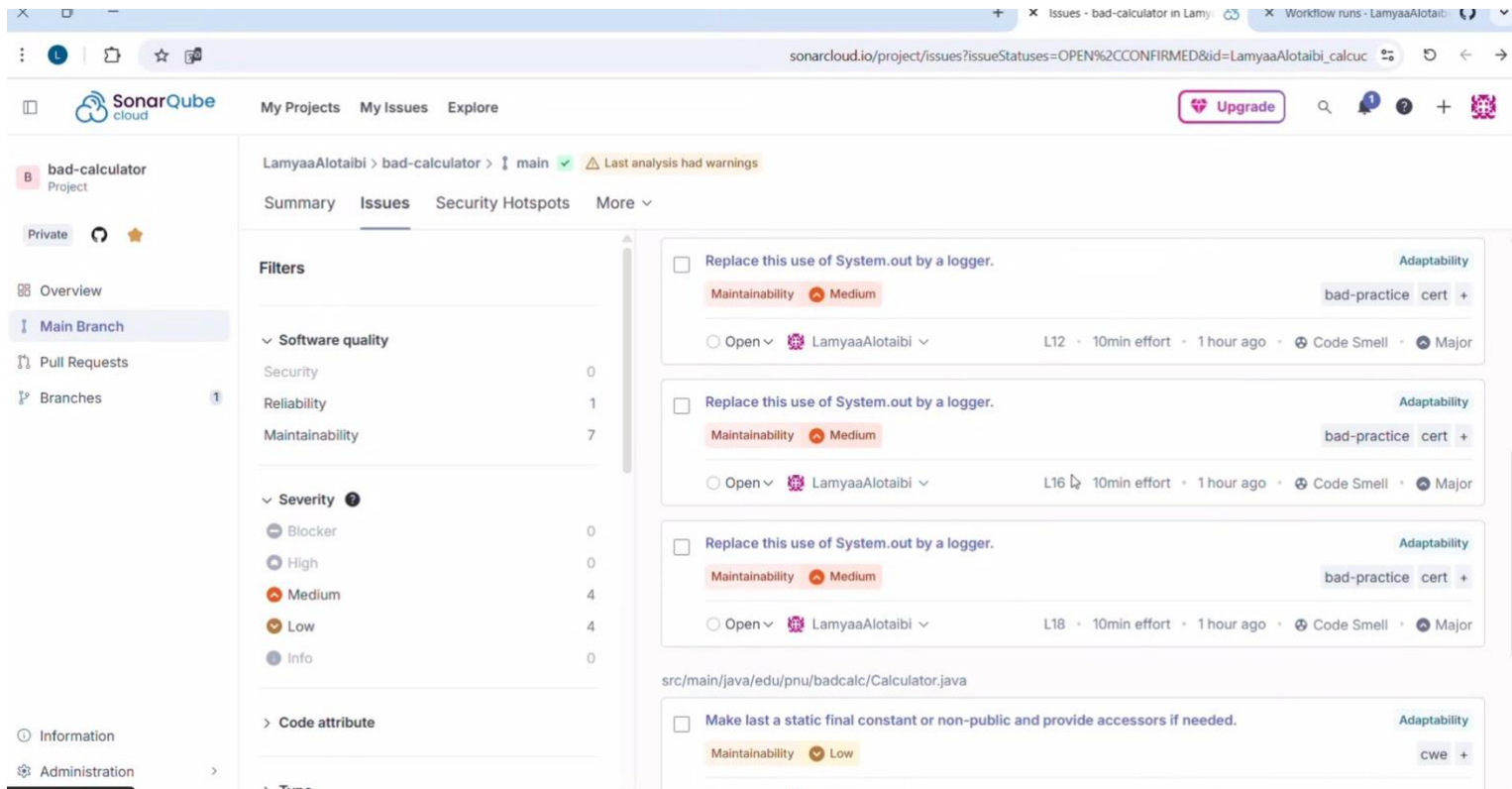
Open LamyaaAlotaibi L10 10min effort 1 hour ago Code Smell Major

Replace this use of System.out by a logger.

Maintainability Medium

bad-practice cert +

Open LamyaaAlotaibi L12 10min effort 1 hour ago Code Smell Major



The 8 issues were mainly:

4× System.out.println misuse, 1× unnecessary import, 1× static field encapsulation issue, 1× type-safety bug, and 1× conditional logging performance issue.

1–4. Using System.out.println instead of a logger (4 issues – Medium severity)

Four separate lines in App.java used System.out.print/println.

SonarQube reports each line as an individual issue.

Why it's bad:

- Not configurable
- Not recommended for production
- Logging frameworks offer levels, formatting, and control

5. Unnecessary import (Low severity)

One line imported a class from the same package.

This import wasn't needed.

Why it's bad:

- Adds noise and reduces clarity

6. Public static field should be final or private (Low severity)

In Calculator.java, the field last was public and non-final.

Why it's bad:

- Breaks encapsulation
- Can be modified from anywhere
- SonarQube suggests making it private or final.

7. Multiply operation needs explicit double cast (Low severity bug)

SonarQube detected a type-safety issue in the multiplication method.

Why it's bad:

- Might cause precision issues
- Considered a minor reliability bug

8. Method should be invoked conditionally (Medium severity)

One of the logging messages in App.java was built even when it wasn't used.

Why it's bad:

- Wastes performance
- String formatting executes even if log level won't print it

B) After fixing (calculator class)

```

1 package edu.pnu.badcalc;
2
3 public class Calculator { 28 usages  Lamyaa Alotaibi
4
5     private double lastResult = 0; 11 usages
6
7     public double add(double a, double b) { 2 usages  Lamyaa Alotaibi
8         lastResult = a + b;
9         return lastResult;
10    }
11
12    public double subtract(double a, double b) { 2 usages  Lamyaa Alotaibi
13        lastResult = a - b;
14        return lastResult;
15    }
16
17    public double multiply(double a, double b) { 2 usages  Lamyaa Alotaibi
18        lastResult = a * b;
19        return lastResult;
20    }
21
22    public double divide(double a, double b) { 3 usages  Lamyaa Alotaibi
23        if (b == 0) {
24            throw new IllegalArgumentException("Cannot divide by zero");
25        }
26        lastResult = a / b;
27        return lastResult;
28    }
29

```

```

30 public long factorial(int n) { 4 usages  Lamyaa Alotaibi
31     if (n < 0) {
32         throw new IllegalArgumentException("Factorial of negative number is undefined");
33     }
34     long result = 1;
35     for (int i = 2; i <= n; i++) {
36         result *= i;
37     }
38     lastResult = result;
39     return result;
40 }

```

```

42     public double parseAndCompute(String expression) { 6 usages  Lamyaa Alotaibi
43         if (expression == null || expression.isEmpty()) {
44             throw new IllegalArgumentException("Expression is empty");
45         }
46
47         expression = expression.trim();
48
49         if (expression.endsWith("!")) {
50             String numStr = expression.substring(0, expression.length() - 1).trim();
51             int n = (int)parseNumber(numStr);
52             return factorial(n);
53         }
54
55         String[] tokens = expression.split( regex: "\\s+");
56         if (tokens.length != 3) {
57             throw new IllegalArgumentException("Expression must be in the form: a op b");
58         }
59
60         double a = parseNumber(tokens[0]);
61         double b = parseNumber(tokens[2]);
62         String op = tokens[1];
63
64         return switch (op) {
65             case "+" -> add(a, b);
66             case "-" -> subtract(a, b);
67             case "*" -> multiply(a, b);
68             case "/" -> divide(a, b);
69             default -> throw new IllegalArgumentException("Unsupported operator: " + op);
70         };
71     }

```

```

71     }
72
73     private double parseNumber(String str) { 3 usages  Lamyaa Alotaibi
74         str = str.trim();
75         if (str.equalsIgnoreCase( anotherString: "last")) {
76             return lastResult;
77         }
78         try {
79             return Double.parseDouble(str);
80         } catch (NumberFormatException ex) {
81             throw new IllegalArgumentException("Invalid number: " + str);
82         }
83     }
84
85     > public double getLastResult() { return lastResult; }
86
87     }
88
89

```

1. Proper Validation and Error Handling

- Division now throws an exception instead of returning Infinity.
- Factorial throws an exception for negative inputs.
- `parseAndCompute()` now validates:
 - Empty or null expressions
 - Wrong format
 - Unsupported operators
 - Invalid numbers

2. Cleaner Parsing Logic

- Introduced a helper method `parseNumber()` to safely convert strings to numbers.
- Added support for using "last" as an input to reuse the previous result.
- Added trimming and strict format checking.

3. Improved Code Structure

- `lastResult` was made private for better encapsulation.
- Added getter `getLastResult()` to safely access it.
- Replaced multiple if statements with a clean switch expression.

4. More Stable and Testable Calculator

- Each operation (add, subtract, multiply, divide) now updates the last result correctly.
- Factorial is computed safely with correct boundaries.
- `parseAndCompute()` now handles:
 - Expressions like "2 + 3"
 - "5 * 7"
 - "10 / 2"
 - "4!"
 - Using "last" as an operand

C) After fixing (App class)

```

package edu.pnu.badcalc;

import java.util.Scanner;
import java.util.logging.Level;
import java.util.logging.Logger;

> public class App {  @ Lamyaa Alotaibi

    private static final Calculator calculator = new Calculator();  1 usage
    private static final Logger logger = Logger.getLogger(App.class.getName());  8 usa

> public static void main(String[] args) {  @ Lamyaa Alotaibi
    logger.info(() -> "Bad Calculator v2.0");
    logger.info(() -> "Supported operations: + - * / and factorial (!)");
    logger.info(() -> "Type 'quit' to exit.");

    try (Scanner scanner = new Scanner(System.in)) {
        while (true) {
            logger.info(() -> "> ");
            String input = scanner.nextLine().trim();

            if (input.equalsIgnoreCase("quit")) {
                logger.info(() -> "Exiting calculator...");
                break;
            }

            try {
                double result = calculator.parseAndCompute(input);
                logger.info(() -> "= " + result);
            } catch (IllegalArgumentException ex) {
                logger.log(Level.WARNING, () -> "Error: " + ex.getMessage());
            } catch (Exception ex) {

```

D) After fixing (Pom)

```

1  <project xmlns="http://maven.apache.org/POM/4.0.0"
2
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4
5      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
6  <modelVersion>4.0.0</modelVersion>
7  <groupId>edu.pnu</groupId>
8  <artifactId>bad-calculator</artifactId>
9  <version>0.0.1</version>
10 <name>bad-calculator</name>
11 <properties>
12 <maven.compiler.source>17</maven.compiler.source>
13 <maven.compiler.target>17</maven.compiler.target>
14 <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 </properties>
16 <dependencies>
17 <dependency>
18 <groupId>org.junit.jupiter</groupId>
19 <artifactId>junit-jupiter</artifactId>
20 <version>5.10.0</version>
21 <scope>test</scope>
22 </dependency>
23 </dependencies>
24 <build>
25 <plugins>
26 <plugin>
27 <groupId>org.jacoco</groupId>
28 <artifactId>jacoco-maven-plugin</artifactId>
29 <version>0.8.10</version>
30 <executions>

```

```

29 <version>0.8.10</version>
30 <executions>
31 <execution>
32 <goals>
33 <goal>prepare-agent</goal>
34 </goals>
35 </execution>
36 <execution>
37 <id>report</id>
38 <phase>test</phase>
39 <goals>
40 <goal>report</goal>
41 </goals>
42 </execution>
43 </executions>
44 </plugin>
45 </plugins>
46 </build>
47 </project>
48

```

- Updated Java version settings

The project was configured to use Java 17 through the `maven.compiler.source` and `maven.compiler.target` properties. This ensures the whole project compiles consistently using the same Java version.

- Added JUnit 5 dependency

A clean JUnit 5 (junit-jupiter) dependency was added under the test scope so the test cases run correctly and integrate with Maven's test lifecycle.

- Added JaCoCo plugin for test coverage

The JaCoCo Maven plugin was included to automatically generate coverage reports:

- prepare-agent attaches the coverage agent before running tests.
- report generates the coverage report during the test phase.

This setup makes SonarCloud able to read the test coverage percentage.

- Organized build/configuration structure

The build section is now cleaner and properly structured with the plugin executions listed in the correct order.

E) Runtime after fixing


```

INFO: Bad Calculator v2.0
edu.pnu.badcalc.App main م ٨:٥٧:٥٣ ٢٠٢٥ ,١٨ نوفمبر
INFO: Supported operations: + - * / and factorial (!)
edu.pnu.badcalc.App main م ٨:٥٧:٥٣ ٢٠٢٥ ,١٨ نوفمبر
INFO: Type 'quit' to exit.
edu.pnu.badcalc.App main م ٨:٥٧:٥٣ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
9
edu.pnu.badcalc.App main م ٨:٥٧:٥٧ ٢٠٢٥ ,١٨ نوفمبر
WARNING: Error: Expression must be in the form: a op b
edu.pnu.badcalc.App main م ٨:٥٧:٥٧ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
/
edu.pnu.badcalc.App main م ٨:٥٨:٠٠ ٢٠٢٥ ,١٨ نوفمبر
WARNING: Error: Expression must be in the form: a op b
edu.pnu.badcalc.App main م ٨:٥٨:٠٠ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
k
edu.pnu.badcalc.App main م ٨:٥٨:٠١ ٢٠٢٥ ,١٨ نوفمبر
WARNING: Error: Expression must be in the form: a op b
edu.pnu.badcalc.App main م ٨:٥٨:٠١ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
4 - 2
edu.pnu.badcalc.App main م ٨:٥٨:٠٩ ٢٠٢٥ ,١٨ نوفمبر
INFO: = 2.0

```

```

INFO: >
2 + 5
edu.pnu.badcalc.App main م ٨:٥٨:١٥ ٢٠٢٥ ,١٨ نوفمبر
INFO: = 7.0
edu.pnu.badcalc.App main م ٨:٥٨:١٥ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
4 * 5
edu.pnu.badcalc.App main م ٨:٥٨:٢٢ ٢٠٢٥ ,١٨ نوفمبر
INFO: = 20.0
edu.pnu.badcalc.App main م ٨:٥٨:٢٢ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
9 / 3
edu.pnu.badcalc.App main م ٨:٥٨:٢٩ ٢٠٢٥ ,١٨ نوفمبر
INFO: = 3.0
edu.pnu.badcalc.App main م ٨:٥٨:٢٩ ٢٠٢٥ ,١٨ نوفمبر
INFO: >
quit
edu.pnu.badcalc.App main م ٩:٠٢:٣٢ ٢٠٢٥ ,١٨ نوفمبر
INFO: Exiting calculator...

```

Process finished with exit code 0

3) Test Case Junit

A. Test Case Code (Annotated with @Test)

```
1 package edu.pnu.badcalc;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.Test;
5
6 class CalculatorTest {  Lamyaa Alotaibi
7
8     @Test  Lamyaa Alotaibi
9     void testAdd() {
10         Calculator c = new Calculator();
11         assertEquals( expected: 5.0, c.add( a: 2, b: 3));
12     }
13
14     @Test  Lamyaa Alotaibi
15     void testSubtract() {
16         Calculator c = new Calculator();
17         assertEquals( expected: 1.0, c.subtract( a: 5, b: 4));
18     }
19
20     @Test  Lamyaa Alotaibi
21     void testMultiply() {
22         Calculator c = new Calculator();
23         assertEquals( expected: 6.09, c.multiply( a: 2.9, b: 2.1), delta: 0.0001);
24     }
25
26     @Test  Lamyaa Alotaibi
27     void testDivideNormal() {
28         Calculator c = new Calculator();
29         assertEquals( expected: 2.0, c.divide( a: 6, b: 3));
30     }
31 }
```

```
31
32     @Test  Lamyaa Alotaibi
33     void testDivideByZero() {
34         Calculator c = new Calculator();
35         assertThrows(IllegalArgumentException.class, () -> c.divide( a: 10, b: 0));
36     }
37
38     @Test  Lamyaa Alotaibi
39     void testFactorialBasic() {
40         Calculator c = new Calculator();
41         assertEquals( expected: 120, c.factorial( n: 5));
42     }
43
44     @Test  Lamyaa Alotaibi
45     void testFactorialZero() {
46         Calculator c = new Calculator();
47         assertEquals( expected: 1, c.factorial( n: 0));
48     }
49
50     @Test  Lamyaa Alotaibi
51     void testFactorialNegative() {
52         Calculator c = new Calculator();
53         assertThrows(IllegalArgumentException.class, () -> c.factorial( n: -4));
54     }
55 }
```

```

55 |
56 | @Test  Lamyaa Alotaibi
57 | void testParseAdd() {
58 |     Calculator c = new Calculator();
59 |     assertEquals( expected: 5.0, c.parseAndCompute( expression: "2 + 3"));
60 | }
61 |
62 | @Test  Lamyaa Alotaibi
63 | void testParseSubtract() {
64 |     Calculator c = new Calculator();
65 |     assertEquals( expected: 1.0, c.parseAndCompute( expression: "5 - 4"));
66 | }
67 |
68 | @Test  Lamyaa Alotaibi
69 | void testParseMultiply() {
70 |     Calculator c = new Calculator();
71 |     assertEquals( expected: 6.09, c.parseAndCompute( expression: "2.9 * 2.1"), delta: 0.0001);
72 | }
73 |
74 | @Test  Lamyaa Alotaibi
75 | void testParseDivide() {
76 |     Calculator c = new Calculator();
77 |     assertEquals( expected: 2.0, c.parseAndCompute( expression: "6 / 3"));
78 | }
79 |

```

```

79 |
80 | @Test  Lamyaa Alotaibi
81 | void testParseInvalidOp() {
82 |     Calculator c = new Calculator();
83 |     assertThrows(IllegalArgumentException.class, () -> c.parseAndCompute( expression: "5 ^ 2"));
84 | }
85 |
86 |

```

The JUnit test cases used to confirm the accuracy of the Calculator functions are displayed in the screenshots. When a method is marked with the `@Test` annotation, JUnit automatically runs it as part of the testing procedure.

These tests cover several calculator behaviors:

- Arithmetic operations: addition, subtraction, multiplication, and division.

Each operation is validated using `assertEquals()` to ensure the returned result matches the expected output.

- Expression parsing: using `parseAndCompute()` to evaluate expressions such as "2 + 3" and "6 / 3".

This confirms that the calculator can correctly interpret and compute user-entered strings.

- Exception handling: using `assertThrows()` to verify that the calculator rejects invalid inputs, including:

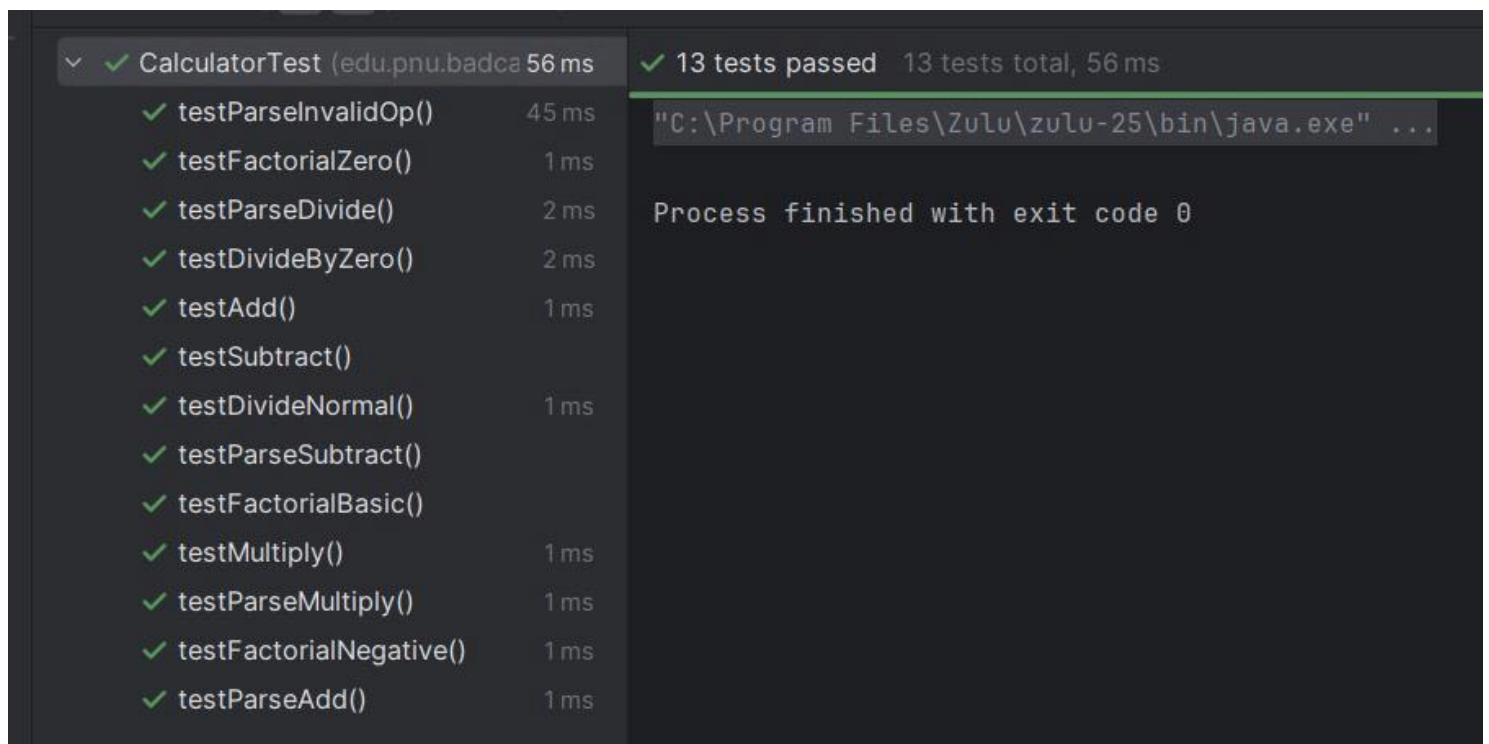
- division by zero
- unsupported operators (e.g., "5 ^ 2")
- invalid factorial input (negative numbers)

- Factorial function: tests include valid positive inputs, the base case (0!), and invalid inputs.

This ensures the method behaves correctly under normal and boundary conditions.

Overall, the code shows that the team tested both edge cases and normal cases to make sure the Calculator functions dependably in a variety of situations.

B. Test Results



The screenshot shows the IntelliJ IDEA test results interface. On the left, a list of 13 tests for `CalculatorTest` is shown, all of which passed. The tests include `testParseInvalidOp()`, `testFactorialZero()`, `testParseDivide()`, `testDivideByZero()`, `testAdd()`, `testSubtract()`, `testDivideNormal()`, `testParseSubtract()`, `testFactorialBasic()`, `testMultiply()`, `testParseMultiply()`, `testFactorialNegative()`, and `testParseAdd()`. The total execution time for all tests is 56 ms. On the right, a summary bar indicates "13 tests passed" and "13 tests total, 56 ms". Below this, a terminal window shows the command `"C:\Program Files\Zulu\zulu-25\bin\java.exe" ...` and the message "Process finished with exit code 0".

Test Name	Duration
✓ CalculatorTest (edu.pnu.badca)	56 ms
✓ testParseInvalidOp()	45 ms
✓ testFactorialZero()	1 ms
✓ testParseDivide()	2 ms
✓ testDivideByZero()	2 ms
✓ testAdd()	1 ms
✓ testSubtract()	1 ms
✓ testDivideNormal()	1 ms
✓ testParseSubtract()	1 ms
✓ testFactorialBasic()	1 ms
✓ testMultiply()	1 ms
✓ testParseMultiply()	1 ms
✓ testFactorialNegative()	1 ms
✓ testParseAdd()	1 ms

✓ 13 tests passed 13 tests total, 56 ms

```
"C:\Program Files\Zulu\zulu-25\bin\java.exe" ...  
  
Process finished with exit code 0
```

The test suite ran successfully after all unit tests were written and executed using JUnit.

The results show that:

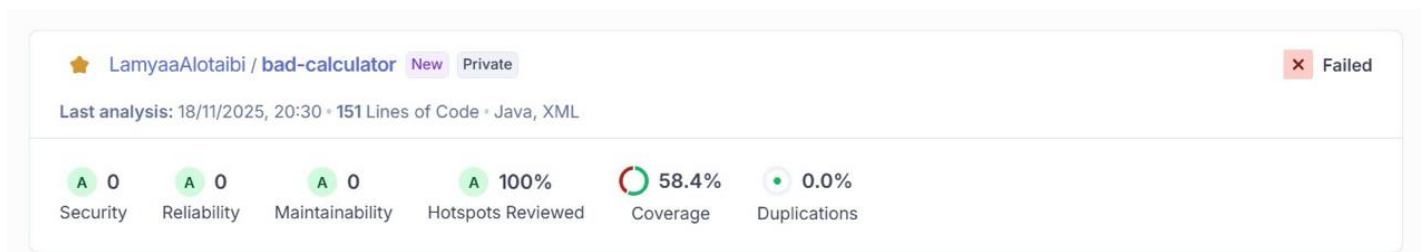
- All 13 out of 13 tests passed
- There were no errors or failures
- Execution finished with exit code 0, confirming that the process completed successfully

This demonstrates that:

- All arithmetic operations (add, subtract, multiply, and divide) operate as intended
- Division by zero is handled correctly using `assertThrows`
- The factorial method correctly accepts positive input and the base case (0!), while rejecting negative values
- The expression parser (`parseAndCompute()`) evaluates valid expressions accurately and rejects invalid ones such as `"5 ^ 2"`

The fully green test bar confirms that the implementation behaves as expected without any exceptions or runtime errors.

4) JaCoCo Coverage



Security

0 Open issues

A

Reliability

0 Open issues

A

Maintainability

0 Open issues

A

Accepted Issues

0

Clock icon

Coverage

58.4%

No conditions set on 66 Lines to cover

Duplications

0.0%

No conditions set on 176 Lines

Security Hotspots

0



After implementing and executing all 13 JUnit test cases, a new SonarQube analysis was performed. The updated report reflects the final state of the project after testing.

Final Coverage Result

- Coverage: 58.4%
- Lines to Cover: 66
- Covered Lines: 36
- Duplications: 0.0%
- Security Issues: 0 (A)
- Reliability Issues: 0 (A)
- Maintainability Issues: 0 (A)
- Security Hotspots: 0
- Quality Gate: Failed (because coverage is below the 80% threshold)

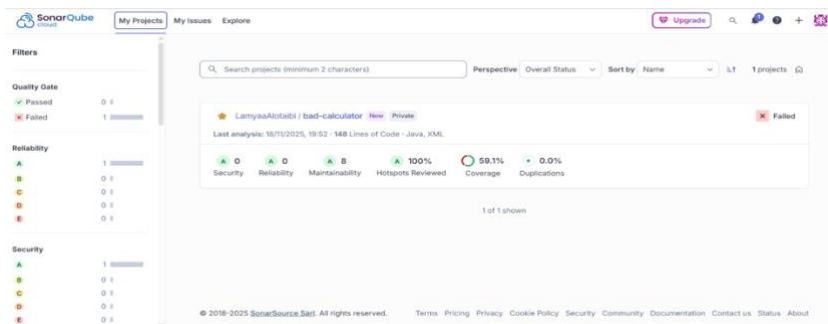
All implemented test cases passed without error, and the test results are still valid and successful despite the slight reduction in overall coverage.

The analysis verifies that the codebase is still clean, stable, and free of errors or vulnerabilities even though the coverage percentage has decreased. SonarQube found no structural or code-quality issues, and all security, dependability, and maintainability metrics received the highest rating (A).

The Quality Gate was marked as "Failed" because the coverage threshold (required $\geq 80\%$) was the only unmet condition. This simply shows that not all code paths are covered by the current test suite; it does not signify a functional or testing failure.

Overall, the revised report shows that every test was completed successfully and that the system operates as intended in every scenario that was tested.

5) SonarCloud dashboard after fixing



The SonarCloud dashboard above provides a view at the project metrics after the improvement.

Reliability (A-0): Measures the likelihood of code failure, and 0 proof that there are no critical Bugs that might cause it.

Security (A-0): Measures how exposed the code is to vulnerabilities, we got 0 and it means we applied good security practices and ensure the code is safe.

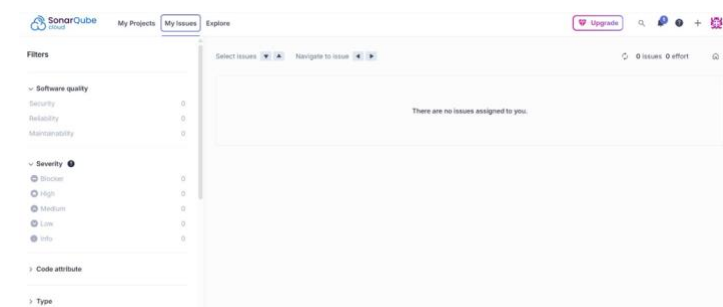
Maintainability (A-8): how easy the code is to understand and change, based on number of code smells, The A rating confirms a good structure, making it easy to work with.

Honeypots reviewed (A-100%): Sections of the code that might require manual check, team has reviewed and confirmed the security in all parts.

Coverage (59.1%): percentage of source code lines that have been Unit Tested, proves that we implemented unit tests.

Duplications (A-0%): percentage of repeated code, 0 indicates no duplicate code included.

A)Issues list status



This image shows that there are no issues currently assigned to our Issues List on the SonarCloud dashboard, confirming that all assigned quality issues have been resolved.

6) Github actions workflow

GitHub Actions Workflow: This image shows that our Continuous Integration workflow is successfully operating. executing the Build, Unit Tests (Tests), and Code Analysis (SonarScan) steps for every push. This process proves good code quality checking process.

test coverage	Build and Analyze with SonarCloud #9: Commit 00a7ab6 pushed by LamyaaAlotaibi	passed	Nov 15, 5:04 AM GMT+3 1m 12s
fix: replace system.out with logger	Build and Analyze with SonarCloud #8: Commit 87f8181 pushed by LamyaaAlotaibi	passed	Nov 15, 5:20 AM GMT+3 1m 10s
fix sonar issues	Build and Analyze with SonarCloud #7: Commit 982072a pushed by LamyaaAlotaibi	passed	Nov 15, 5:23 AM GMT+3 1m 3s
after test	Build and Analyze with SonarCloud #6: Commit 906b763 pushed by LamyaaAlotaibi	passed	Nov 15, 5:11 AM GMT+3 1m 2s
update	Build and Analyze with SonarCloud #5: Commit a84a4d1 pushed by LamyaaAlotaibi	passed	Nov 15, 4:45 AM GMT+3 1m 15s
with test	Build and Analyze with SonarCloud #4: Commit a50c57e pushed by LamyaaAlotaibi	passed	Nov 15, 4:34 AM GMT+3 1m 5s
Merge branch 'main' of https://github.com/LamyaaAlotaibi/calcul	Build and Analyze with SonarCloud #2: Commit a5589ef pushed by LamyaaAlotaibi	passed	Nov 15, 1:28 AM GMT+3 1m 5s
Add SonarCloud CI workflow for build and analysis	Build and Analyze with SonarCloud #1: Commit a52b60d pushed by LamyaaAlotaibi	passed	Nov 15, 1:24 AM GMT+3 1m 4s

All workflows
Showing runs from all workflows

Filter workflow runs

13 workflow runs	Event	Status	Branch	Actor
lambda	Build and Analyze with SonarCloud #15: Commit a6aef53 pushed by LamyaaAlotaibi	passed		23 minutes ago 1m 0s
about prinitn statement	Build and Analyze with SonarCloud #14: Commit 87f8a0d pushed by LamyaaAlotaibi	passed		31 minutes ago 50s
App	Build and Analyze with SonarCloud #13: Commit 00a448a pushed by LamyaaAlotaibi	passed		Today at 7:52 PM 1m 7s
Last test	Build and Analyze with SonarCloud #12: Commit a65c6d8 pushed by LamyaaAlotaibi	passed		Today at 7:47 PM 1m 0s
after test 2	Build and Analyze with SonarCloud #10: Commit 7b11724 pushed by LamyaaAlotaibi	passed		Nov 15, 6:09 AM GMT+3 1m 0s

7) Discussion

The Calculator App's quality has clearly improved, according to the SQA process' findings. Thirteen JUnit test cases were implemented after the code was refactored, and all of the tests passed without any errors or failures, demonstrating that the parsing logic, exception handling, and arithmetic operations function as intended for the tested scenarios.

Meanwhile, the final SonarCloud and JaCoCo reports showed that 36 out of 66 lines were covered, with an overall coverage of about 58–59%. This indicates that some parts of the codebase particularly alternative branches and less common error paths remain untested even though the main logic is validated and stable.

The project concluded in a very strong state from the standpoint of static analysis. Every SonarCloud issue that had been found earlier, such as needless imports, inadequate encapsulation, type-safety alerts, and logging issues, was fixed. The final SonarCloud dashboard has A ratings for dependability, security, and maintainability and displays no bugs, vulnerabilities, code smells, or duplication. This demonstrates that the refactoring greatly enhanced readability, structure, and long-term maintainability.

Despite these encouraging outcomes, SonarCloud's Quality Gate remained "Failed" since the necessary $\geq 80\%$ coverage threshold was not reached. Crucially, this only indicates that the current test suite falls short of the coverage level specified by the quality profile; it does not point to functional problems with the system.

8) Conclusion

This project gave us the chance to investigate how useful tools, as opposed to conjecture, can be used to assess software quality. Working with the provided Calculator App allowed us to find actual problems in the code, comprehend how they impacted correctness and maintainability, and implement fixes through testing and refactoring.

Whether it was better logic, less code smells, or more coverage, using JUnit, JaCoCo, and SonarCloud allowed us to clearly see the effects of our changes. We now have a better understanding of how quality assurance tools facilitate dependable software development and how minor code modifications can have a big impact on overall quality thanks to this process. All things considered, the project deepened our comprehension of SQA principles and provided us with practical experience implementing them on an actual codebase.

9) References

1. Howard T. Garst Smith, Software Quality Assurance: A Guide for Developers and Auditors, CRC Press, 2020.
2. Software Quality Assurance: A Self-Teaching Introduction, Mercury Learning, 2018.
3. SonarCloud Documentation – <https://sonarcloud.io>
4. JaCoCo Documentation – <https://www.jacoco.org>
5. GitHub Actions Documentation – <https://docs.github.com/en/actions>